

UART Transceiver

This project implements a complete UART (Universal Asynchronous Receiver/Transmitter) transceiver in Verilog, including both the transmitter (TX) and receiver (RX) modules along with a loopback integration for end-to-end testing. It is designed for use with a 100 MHz system clock and standard UART framing. In asynchronous UART communication, data is sent as serial “frames” of bits without a shared clock signal. Each frame typically starts with a **start bit** (logic 0), followed by a fixed number of **data bits** (e.g. 8 bits), optional **parity bit**, and ends with one or more **stop bits** (logic 1). The line idles at logic 1 when no data is sent. The Verilog design uses finite-state machines to generate and recognize these bits: the TX FSM drives the output line low and high for each bit, and the RX FSM waits for a start-bit transition and then samples the incoming bits at their midpoints.

Features:

- **8-bit data, no parity, 2 stop bits (8N2)** (configurable). Each frame: 1 start bit (0), 8 data bits (LSB first), 2 stop bits .
- **Baud rate generation:** A clock divider (CLKS_PER_BIT) sets the bit timing. For example, with 100 MHz clock and 9600 baud, $\text{CLKS_PER_BIT} \approx 100\text{MHz}/9600 \approx 10417$ cycles.
- **Modular design:** Separate `uart_transmitter.v` and `uart_receiver.v` modules, plus a `uart_loopback.v` top-level that ties TX output to RX input for testing.
- **FSM-based design:** Both TX and RX use clear state machines (Idle, Start, Data, Stop, Done).
- **Fully verified:** Testbenches `transmitter_tb.v`, `receiver_tb.v`, and `uart_loopback_tb.v` simulate and verify operation.

Architecture Overview

The design has three main blocks (Transmitter, Receiver, Baud-Rate Clock). The UART Transmitter module loads an 8-bit data word and shifts it out serially on the tx line (preceded by a start bit 0 and followed by a stop bit 1). The UART Receiver monitors the rx line, detects the falling edge of the start bit, and then samples each data bit (in the middle of its bit period) to reconstruct the byte. Both modules share the same 100 MHz clock and use a divider (baud counter) to meet the target baud rate. The bullets below summarize the key blocks:

- **Baud-Rate Clock (Divider):** A counter driven by the 100 MHz system clock divides it down to the desired baud rate (e.g. 9600 bps). This clock sets the timing for each bit in both the Transmitter and Receiver.

- **Transmitter (FSM):** Implements four states (IDLE → START → DATA → STOP). When tx_start is asserted in IDLE, it loads data_in into a shift register. It then drives tx low for the start bit, shifts out each data bit (LSB first) on successive baud pulses, and finally drives tx high for the stop bit. The busy output remains high until the entire frame is sent.
- **Receiver (FSM):** Also uses IDLE → START → DATA → STOP states. In IDLE it waits for rx to go low. On detecting the start bit, it waits half a bit period and then samples each of the 8 data bits at the baud rate. After the stop bit, it outputs the received byte on data_out and raises rx_done to indicate a complete frame.
- **UART Loopback (Top Module):** Instantiates the Transmitter and Receiver modules and wires tx → rx for end-to-end testing. A start pulse triggers the Transmitter to send tx_data, and the Receiver captures it back into rx_data. This setup lets us verify that any byte sent out is correctly received looped back.

Development & Verification Workflow

We followed a structured design and verification process, verifying each part before moving on:

- **Module Implementation:** Write Verilog RTL for each block (transmitter.v and receiver.v) based on the UART frame and timing requirements. Each block is coded as an FSM with the states described above.
- **Testbench Creation:** Develop a Verilog testbench for each module (transmitter_tb.v, receiver_tb.v). Each testbench instantiates the module under test, drives clocks/resets, and applies input stimulus. In HDL design, the testbench is a separate top-level module that feeds inputs to the DUT and checks outputs. This allows us to simulate and verify functionality before hardware.
- **Simulation and Debug:** Compile and run the simulations (using Vivado's simulator, ModelSim, or Icarus Verilog). We examined waveform outputs to confirm correct behavior. For example, in the transmitter waveform, tx goes low for the start bit and then toggles for each data bit; in the receiver waveform, the bits on rx are sampled and reassembled. This follows the standard flow – write RTL, write testbench, compile, run simulation, and view waveforms to catch bugs early.
- **Integration (Loopback) Test:** Use the uart_loopback_tb.v testbench to send data through the full system. We apply a sample byte on tx_data, assert start, and then check that rx_data matches the original byte. This end-to-end test confirms that the transmitter and receiver work correctly together.
- **Documentation:** Draw and include diagrams (UART frame format, FSM state diagrams) and add descriptive comments. Good documentation helps others understand the design's structure and usage.

Using the Design / Simulation

- **AMD Vivado :** Create a project and add all .v source files and testbenches. Run transmitter_tb to verify the transmitter behavior (observe tx and busy), run receiver_tb by driving a bit pattern on rx, and run uart_loopback_tb for end-to-end verification. In Vivado, the waveform viewer shows the signals over time for debugging. (I Used for this Project)
- **FPGA Implementation (Upcoming):** The design parameters (baud_rate, clk_freq) can be adjusted for real hardware. To run on a board (Basys3), map the tx/rx pins to actual FPGA pins, and communicate with a PC via a USB-UART adapter. The loopback mode can be tested by connecting the TX and RX pins externally as well.

Sources: Standard UART design principles and verification methods were used to inform this documentation.
